Program-Guided Temporal Reasoning over Routed Video Evidence for TimeLogicQA

Yuyang Sun[1] Yu Zhang[2] Xiaogang Wang[3] Wenwu He[2] Yuxia Chen[4]
[1]Southeast University [2]Freedo Technology [3]Southwest University [4]Independent Researcher

Abstract

Temporal video question answering is fundamentally an evidence organization problem: a correct response may depend on when an event occurs, how long it persists, or whether two actions intersect. For TimeLogicQA, we develop a program-guided inference pipeline that assigns different evidence budgets according to clip duration and the temporal operator expressed by the question. A lightweight parser identifies the target events, answer format, candidate responses, and required relation. Short clips are represented by ordered full-frame observations, whereas long clips are narrowed to event-centered temporal windows. A multimodal model converts these views into structured observations; interval verifiers and an operator-specific reducer then determine the answer from explicit temporal relations. The system retains a prediction only when the visual observations, symbolic reduction, and confidence checks are mutually consistent, limiting unsupported answer changes. Without additional task-specific training, the resulting system reaches an AvgAcc of 81.8 on the official test evaluation.

1. Introduction

Two videos can contain the same people, objects, and actions while supporting different answers solely because those events unfold in a different order. This makes temporal video question answering distinct from ordinary action recognition: recognizing what appears is not enough; a system must also determine when it happens and how separate events relate. TimeLogicQA isolates this requirement through questions about occurrence, precedence, repeated-event constraints, boundary conditions, and temporal overlap [2].

The main difficulty is not confined to a single model capability. A system can fail while locating a brief event, lose chronological information when compressing a clip, or apply the wrong rule after the relevant frames have already been found. Quantified statements such as always-before require checking multiple occurrences, while until and overlap questions depend on event boundaries rather than isolated frames. Direct prompting tends to entangle these stages and can substitute a plausible language prior for evidence that is actually visible in the video.

The official test split contains 3,000 questions with Boolean and four-way multiple-choice answers. Its operators cover several qualitatively different forms of reasoning, so we use the operator and answer mode as routing signals rather than treating every example identically.

Clip duration creates a second source of asymmetry. In a short, action-dense video, sparse sampling may remove the only frames that establish an ordering or boundary. Processing every frame of a long video, however, adds cost and surrounds the decisive moments with irrelevant context. Effective reasoning therefore requires both temporal coverage and selective attention, but the appropriate balance changes with the input.

We cast each question as a compact temporal program and organize inference around the evidence needed to execute it. The question parser extracts events, candidates, answer mode, and operator. A routing stage then preserves ordered global evidence for short clips and searches event-focused windows in long clips. The multimodal model serves as an observation generator rather than the final temporal judge: its outputs are converted into structured event evidence, dense intervals are recovered when necessary, and deterministic operator rules produce the answer.

This design contributes three practical elements: duration- and operator-aware evidence routing, explicit interval-based verification, and conservative fusion across visual, programmatic, and confidence signals. Together they separate perception from temporal decision making and reduce answer changes that are not supported by consistent evidence.

2. Method

2.1. Overview

Figure 1 shows the overall Temporal Evidence Routing pipeline. The system has six stages: question parsing, event canonicalization, duration-aware visual evidence construction, structured VLM evidence generation, temporal micro-program verification, and temporal reduction with conservative fusion.

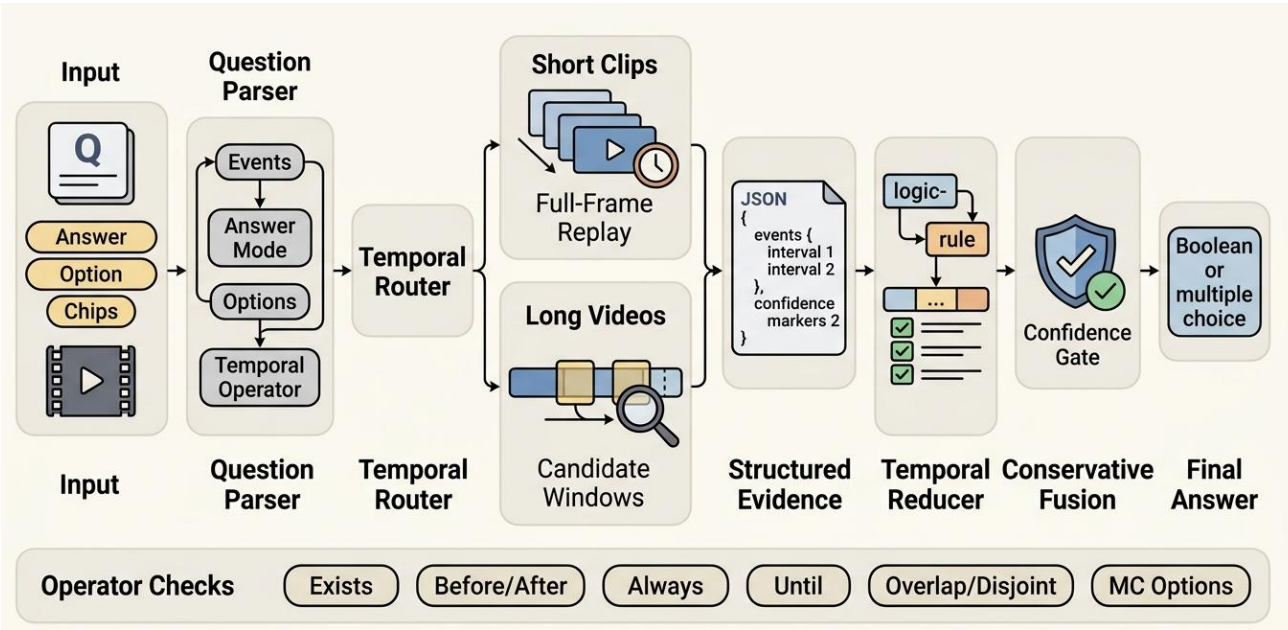


Figure 1. Temporal Evidence Routing. The pipeline parses the question, routes the video input, obtains structured event evidence from targeted visual context, and applies temporal rules before final fusion.

2.2. Question Parsing and Operator Routing

For each question, we identify the answer mode, event phrases, candidate options, and temporal operator. Multiple-choice questions are decomposed into independent option checks instead of being treated as a single global classification problem. This is important because an option may be visually present but still fail the requested temporal relation.

The parser maps each question into a compact program. A Boolean before question becomes a relation between two event variables, while a multiple-choice always-before question becomes four option-specific programs that share the same temporal condition. The parsed operator also controls the evidence budget. Existence questions can be answered from coarse evidence, while always, until, and overlap questions require more careful occurrence lists and counterexample checks.

2.3. Event Canonicalization

The parsed event phrases are normalized before visual reasoning. We separate action predicates from object arguments, map surface forms such as *holding*, *taking*, *putting*, *opening*, and *closing* into transition-aware predicates, and attach each phrase to its role in the temporal program. This step prevents the reducer from treating semantically similar but temporally different states as equivalent.

For ambiguous verbs, the system builds strict visual definitions. A *take* event requires a transition into possession, a *put* event requires release to a support or location, and an *open* or *close* event requires a visible state transition rather than merely seeing the object. These definitions are used both in the VLM prompt and in later confidence checks.

2.4. Visual Evidence Construction

The visual input is selected according to video duration and temporal difficulty.

- **Short clips.** For clips under a few seconds, we decode the clip into an ordered frame sequence and provide the frames as a slow replay. The frame index becomes the temporal axis, which avoids losing transient actions through sparse sampling.
- **Long videos.** For longer videos, we build event-focused candidate windows. A frame bank is sampled over the full video, visual-text similarity retrieves candidate moments for each event phrase, and neighboring frames are merged into temporal windows. The VLM only receives the windows that are relevant to the parsed events and options.

This routing reduces the burden on the VLM. Short clips preserve dense temporal changes, while long videos avoid asking the model to search an entire procedure for a small set of events. In the long-video retrieval branch, we use OpenAI CLIP ViT-B/16 [1] and google/siglip-base-patch16-224 [4] as local candidate-window selectors, followed by a stronger multimodal model for visual verification.

2.5. Structured Visual Evidence

The VLM is prompted to classify and describe events rather than directly answer the input question. Each event is classified as present, not visible, or uncertain, with frame or window-level evidence. For repeated actions, the model must list separate occurrences. For ambiguous action verbs such as *hold*, *take*, *put*, *open*, and *close*, the prompt uses strict visual definitions so that result states and direct transitions are not conflated.

The structured output contains:

- event status and confidence,
- one or more occurrence intervals or ordered frame ranges,
- short visual evidence for each event,
- near-miss evidence when an event is visually similar but not exact.

This evidence-first prompting follows the observation that modern VLMs are strong visual recognizers but can be unstable when asked to combine perception, event localization, and temporal logic in one step. We therefore use the multimodal model primarily as an evidence generator.

2.6. Temporal Micro-Program Verification

For dense short clips, the system adds a second verifier after structured evidence extraction. The verifier constructs a compact set of action instances from the ordered frame replay: each instance stores the canonical predicate, visible object arguments, frame range, and confidence. It then executes the parsed temporal program directly over this instance list.

This stage is especially useful for state-transition actions. Global VLM evidence may say that a person is holding an object, but the temporal program may require whether the person *took* it before another event or merely held it afterward. The micro-program verifier therefore distinguishes persistent states from transition events and rejects near misses. Its output is not used as an unconditional replacement; it becomes a candidate answer with an explicit confidence and a reason code.

2.7. Temporal Reduction

After evidence extraction, a deterministic reducer applies the operator semantics. The reducer uses the extracted occurrence order and intervals, not the wording order of the question. The core rules are:

- **Before / after:** compare the ordered occurrences of the two events.
- **Always before / after:** verify that the relation holds across all supported occurrences and reject visible counterexamples.
- **Overlap / disjoint:** compare event intervals and their intersections.
- **Until:** identify the maintained event and the boundary event, then check whether the evidence supports the boundary relation.
- **Multiple choice:** evaluate each option independently and choose the option that satisfies the temporal program.

If the evidence is missing or inconsistent, the reducer marks the case as unresolved instead of forcing a low-confidence answer. This is especially important for always and overlap questions, where a single missed occurrence can reverse the answer.

2.8. Conservative Fusion

The final stage combines the base prediction with candidate answers from the routed evidence pipeline. A candidate replacement is accepted only when the event evidence, temporal reducer, answer confidence, and micro-program reason code are consistent. Otherwise, the system keeps the base prediction. This conservative policy is designed to improve high-confidence temporal errors without introducing many noisy changes.

3. Experiments and Result

3.1. Setup

The system is inference-only and does not fine-tune model weights. The main multimodal model used in the final run is gemini-3.1-pro-preview [3]. Structured evidence extraction is generated in JSON mode with temperature 0.0, max_output_tokens=4096, and thinking_level=MEDIUM. The final answer reducer also uses JSON mode with temperature 0.0, max_output_tokens=1024, and thinking_level=LOW. For video input, clips shorter than 2.0 seconds are decoded into all frames and passed as a chronological slow replay; clips from 2.0 to 4.0 seconds are represented by 12 uniformly sampled frames; longer videos use operator-dependent video sampling metadata.

The local retrieval branch first builds a frame bank for each long video, with 8 fps for videos under 10 seconds, 1-second stride for 10–60 second videos, and 2-second stride for videos at least 60 seconds long. Frames are resized to a maximum width of 640 pixels. We then encode every frame and each parsed event phrase with two retrieval backbones: OpenAI CLIP ViT-B/16 [1] and google/siglip-base-patch16-224 [4]. Image and text features are L2-normalized, and frame relevance is scored by the dot product between image and text embeddings. Each backbone returns the top 32 frames for each event. The two rankings are fused by reciprocal-rank fusion with $k = 60$, and the top 8 fused frames are expanded into candidate windows. Each selected timestamp is expanded by ± 6 seconds, windows within 2 seconds are merged, long windows are capped at 20 seconds, and at most 16 windows are passed to the VLM. For always and until questions, three additional coverage windows centered at 15%, 50%, and 85% of the video are added. Confidence values are clipped to $[0, 1]$, and

Table 1. Official test result.

System	AvgAcc
Temporal Evidence Routing	**81.8**

Table 2. Gemini-only baseline versus the final system.

Configuration	AvgAcc	Gain
Gemini-only direct prompting	62.50	–
Temporal Evidence Routing	**81.8**	**+19.30**

unresolved evidence or evidence below the 0.55 confidence threshold is handled conservatively.

3.2. Official Test Result

Table 1 reports the final official test metric. We include only the final verified score.

3.3. Ablations

Table 2 compares direct Gemini prompting with the final Temporal Evidence Routing system. The baseline asks the multimodal model to answer from the video context, while our final system decomposes the task into evidence extraction, temporal micro-program verification, deterministic reduction, and conservative fusion. The comparison isolates the value of explicit temporal reasoning around the same multimodal backbone.

3.4. Observations

The most important empirical observation is that temporal errors are concentrated in operator-specific patterns rather than uniformly distributed across the test set. Direct full-video prompting is often sufficient for simple existence and coarse ordering questions, but it is less reliable for repeated events, all-occurrence constraints, and overlap relations. Separating event evidence from temporal reduction improves control over these cases.

Short clips benefit from full-frame replay because the temporal signal is compressed into very few seconds. Longer videos benefit from event-focused retrieval because the relevant events may occupy only a small part of the video. Multiple-choice questions require independent option checks; otherwise, the model may choose an option because it is visible or semantically plausible even when it violates the temporal relation. The large gap over direct Gemini prompting shows that the main bottleneck is not raw visual recognition alone, but converting visual observations into the exact temporal relation requested by the question.

4. Conclusion

We presented Temporal Evidence Routing, an inference-only approach for temporal-logic video QA. The system decomposes the task into question parsing, routed visual evidence construction, structured event evidence generation, deterministic temporal reduction, and conservative answer fusion. This design reduces reliance on global video impressions and makes the reasoning path more aligned with the temporal operators in TimeLogicQA. The final system achieves an official AvgAcc of 81.8 on the test evaluation.

References

[1] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021. 2, 3

[2] Sirnam Swetha, Hilde Kuehne, and Mubarak Shah. Timelogic: A temporal logic benchmark for video qa. *arXiv preprint arXiv:2501.07214*, 2025. 1

[3] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. 3

[4] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 11975–11986, 2023. 2, 3